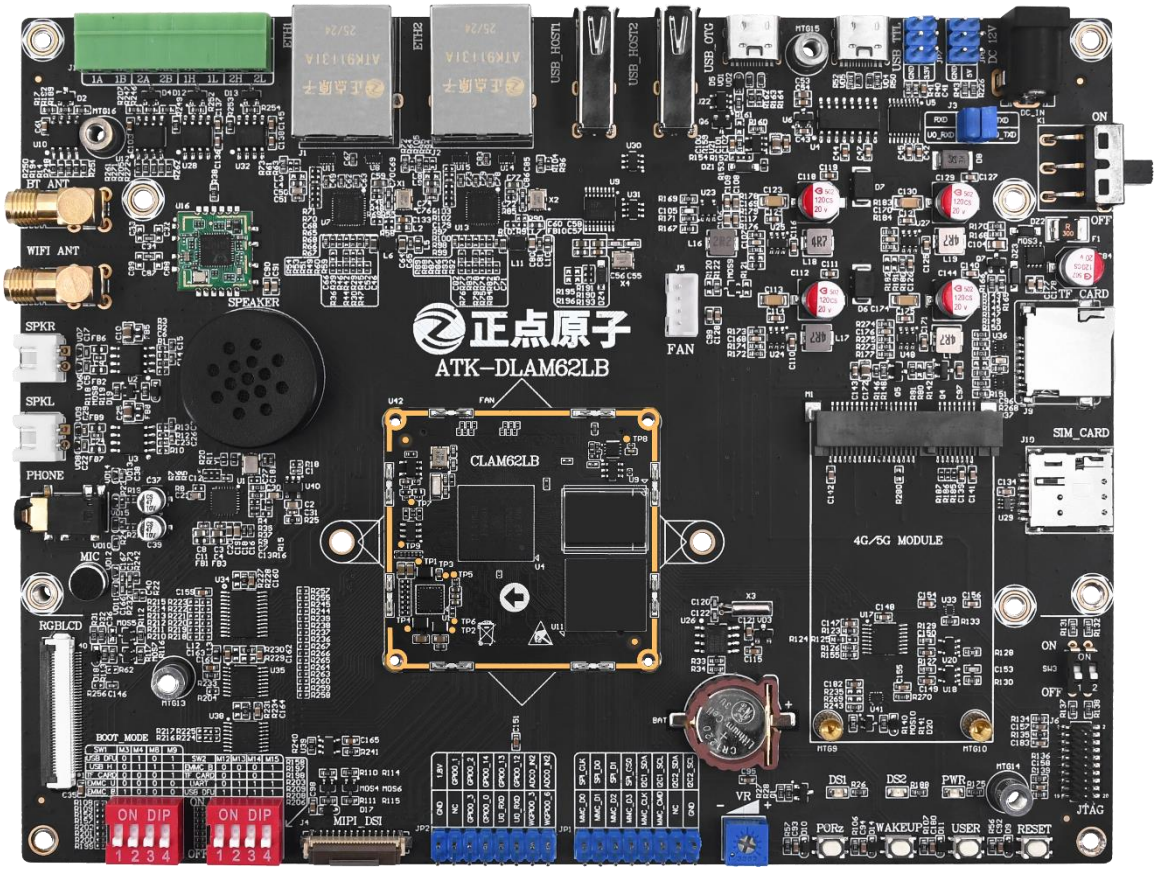
ATK-DLAM62L 出厂系统管脚复用修改手册 V1.0



修订历史:

版本	日期	修改内容
V1.0	2026/3/1	1，第一次发布

正点原子

正点原子公司名称 ：广州市星翼电子科技有限公司

正点原子旗舰店 ：https://zhengdianyuanzi.tmall.com

正点原子官方网站 ：https://www.alientek.com

正点原子资料下载 ：www.openedv.com/docs/index.html

正点原子 B 站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请关注正点原子公众号，资料发布更新我们会通知。

请下载原子哥 APP，数千讲视频免费学习，更快更流畅。

扫码关注正点原子公众号

目录

前言 1
第一章 核心板管脚定义说明 2
1.1 AM62L 芯片管脚定义说明 2
1.2 ATK-CLAM62L 核心板接口定义说明 3
第二章 管脚复用说明 5
2.1 AM62L 管脚复用说明 5
2.2 底板原理图+核心板接口数据手册查询复用功能 6
2.2.1 根据管脚号查找 6
2.2.2 根据功能查找 6
2.3 在出厂系统中查询管脚使用功能 7
第三章 环境搭建和基础操作 9
3.1 开发板基础使用 9
3.2 出厂源码编译 9
3.3 固件更新 9
第四章 出厂系统外设资源 10
4.1 开发板板载外设资源 10
4.2 出厂系统内核源码外设资源 10
【外设复用篇说明】 12
第五章 开发板 GPIO 接口复用 13
5.1 修改引脚为 GPIO 功能 13
5.1.1 添加 GPIO 功能 13
5.1.2 添加 pinctrl 配置 13
5.1.3 功能验证 14
第六章 UART 复用 15
6.1 UART5 15
6.1.1 去掉原本复用功能 15
6.1.2 设备树中添加 UART5 配置 15
6.1.3 功能测试 17
第七章 IIC 接口复用 20
7.1 I2C3 20
7.1.1 去掉原本功能 20
7.1.2 设备树中添加 I2C3 配置 21
7.1.3 功能测试 22
第八章 SPI 接口复用 24
8.1 SPI0 24
8.1.1 去掉原本功能 24
8.1.2 设备树中添加 SPI0 配置 24
8.1.3 功能测试 27
第九章 PWM 接口复用 30
9.1 EHRPWM0_A 30
9.1.1 去掉原本功能 30

9.1.2 设备树中添加 EHRPWM0_A 配置 31
9.1.3 功能测试.....32
第十章 ADC 接口复用 35
10.1 ADC 35
第十一章 CAN 接口复用 36
11.1 CAN1 36
11.1.1 去掉原本功能.....36
11.1.2 设备树中添加 CAN1 配置 36
11.1.3 功能测试.....38

前言

免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

第一章 核心板管脚定义说明

1.1 AM62L 芯片管脚定义说明

ATK-CLAM62L 核心板管脚定义文档路径：

开发板光盘 A 盘-基础资料→06、硬件资料→01、核心板资料→01、核心板管脚定义→ATK-CLAM62LB 核心板接口数据手册.xlsx

在该文档中可以看到芯片管脚当前功能(出厂系统默认配置及功能描述)、管脚号和管脚可复用功能，如下图所示：

核心板管脚号	芯片引脚号	核心板默认配置	核心板信号长度	参考电平	可复用功能(需结合具体需求分析)
A_1	\				
A_2	\				
A_3	\				
A_4	\				
A_5	\				
A_6	\				
A_7		GND			
A_8		GND			
A_9	T23	MMC2_DAT2		3.3V	MMC2_DAT2\MCASP1_AXR2\UART5_TXD\GPIO0_46
A_10	R22	MMC2_DAT3		3.3V	MMC2_DAT3\MCASP1_AXR3\UART5_RXD\GPIO0_45
A_11	U23	MMC2_CMD		3.3V	MMC2_CMD\MCASP1_AFSR\MCASP1_AXR4\UART6_TXD\GPIO0_50
A_12	R23	MMC2_CLK		3.3V	MMC2_CLK\MCASP1_ACLKR\MCASP1_AXR5\UART6_RXD\GPIO0_49
A_13	U22	MMC2_DAT0		3.3V	MMC2_DAT0\MCASP1_AXR0\GPIO0_48
A_14	T22	MMC2_DAT1		3.3V	MMC2_DAT1\MCASP1_AXR1\GPIO0_47

图 1.1.1 ATK-CLAM62L 核心板接口数据手册部分示例

BTB 管脚号：开发板底板 BTB 接口封装库对应的管脚序号。

芯片引脚号：管脚连接 AM62L 处理器具体焊球的编号，方便用户对照 AM62L 数据手册。

核心板默认配置：开发板底板原理图中 BTB 座子的对应管脚名，名字对应的是出厂系统默认功能。例如 T23 管脚，出厂系统默认用做 MMC2_DAT2 信号管脚。

参考电平：管脚对应的信号电压上限大小，有 3.3V 和 1.8V 两种。

可复用功能(需结合具体需求分析)：管脚理论上可以配置的功能，在进行项目评估时可以参考，软件工程师在修改管脚功能时可以参考，硬件工程师在分配原理图管脚功能时可以参考。

ATK-CLAM62L 核心板上默认烧写了一个出厂系统，里面已经配置好管脚的默认功能，比如 C13 管脚在出厂系统上被当做 UART0_TXD 功能，可以结合 ATK-DLAM62L 开发板直接使用。

1.2 ATK-CLAM62L 核心板接口定义说明

在 ATK-DLAM62L 开发板板底板原理图的 U42 就是核心板 BTB 连接器对应的管脚定义，此连接器是 2 个 2x60PIN 的规格，一共有 240 个管脚。核心板背面有管脚序号丝印，如下所示：

图 1.2.1 核心板背面

可以看到核心板背面上有 2 个 BTB 连接器公座，一边的是 A，另一边是 B，和底板上的 BTB 母座管脚号一一对应。

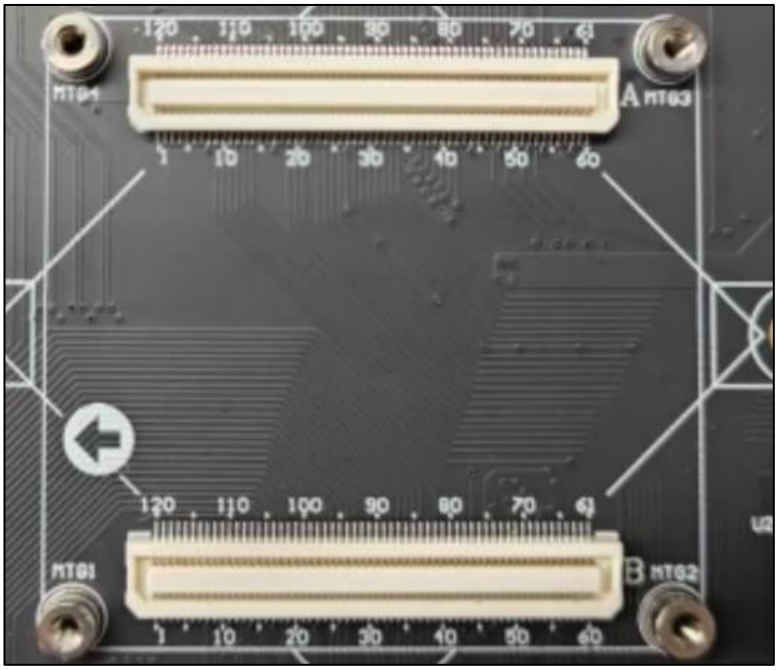


图 1.2.2 开发板底板 BTB 接口

具体管脚功能定义，可以查看开发板底板原理图 U42 部分。

ATK-DLAM62L 底板原理图路径：开发板光盘 A 盘-基础资料→开发板原理图→底板原理图→ATK-DLAM62LB.pdf

在原理图中，只写了核心板在底板上使用的功能（也就是默认出厂系统功能），如果想要查看管脚可复用功能，可以在搜索对应的 IO 编号，在芯片管脚中查看可复用功能。

以 B_45 管脚为例，管脚编号为 C13，默认出厂系统配置功能为 UART0_TXD，现在我们不需要 UART0_TXD 功能，想把它改成其他功能。在《ATK-DLAM62L 核心板接口数据手册.xlsx》中搜索关键词 C13，在可复用部分可以看到如下部分：

B_45	C13	UART0_TXD		3.3V	UART0_TXD\ECAP2_IN_APWM_OUT\SPI2_D1\EHRPWM2_B\GPIO0_93

图 1.2.3 C13 管脚可复用功能

可以看到 C13 这个管脚可以用做功能如下：

UART0_TXD\ECAP2_IN_APWM_OUT\SPI2_D1\EHRPWM2_B\GPIO0_93

那么我们在设计底板之前，就可以考虑将 C13 用做上面其中一个功能，比如我们开发板上就用做 UART0_TXD，用户在用核心板设计产品的时候，可以将它引出，用做 GPIO0_93 或 EHRPWM2_B 等自己需要的功能。

所以一般建议开发板底板上的功能是自己方案需要的话，尽量按照开发板底板默认的功能定义保留。

一般来说，ENET 网口、SD(TF card)接口等专用外设接口管脚、RESET 等系统信号接口管脚尽量不要改动，降低系统修改难度，加快开发进度。

第二章 管脚复用说明

2.1 AM62L 管脚复用说明

在 AM62L 芯片数据手册里找到“5.2 Pin Attributes”章节，此章节是 GPIO 寄存器描述。由于 GPIO 编号在 AM62L 中是唯一的，因此我们可以通过搜索芯片引脚序号来查找对应的管脚作为切入点。

比如笔者搜索，找到对应的配置如下：

Table 5-1. Pin Attributes (ANB, ANQ Packages) (continued)

ANB BALL NUMBER [1]	ANQ BALL NUMBER [1]	BALL NAME [2] PADCONFIG Register [15] PADCONFIG Address [16]	SIGNAL NAME [3]	MUX MODE [4]	TYPE [5]	DSIS [6]	BALL STATE DURING RESET (RX/TX/PULL) [7]	BALL STATE AFTER RESET (RX/TX/PULL) [8]	MUX MODE AFTER RESET [9]	I/O OPERATING VOLTAGE [10]	ANB POWER / ANQ POWER [11]	HYS [12]	BUFFER TYPE [13]	PULL UP/DOWN TYPE [14]
L21	NA	GPMC0_CLK PADCONFIG: PADCONFIG46 0x040840B8	GPMC0_CLK	0	O		Off / Off / Off	Off / Off / Off	7	1.8V / 3.3V	VDDSHV0 / NA	Yes	LVCMOS	PU/PD
			VOUT0_DATA16	1	O									
			MCASP1_AXR3	2	IO	0								
			GPMC0_FCLK_MUX	3	O									
			EHRPWM1_B	4	IO	0								
			TRC_DATA6	6	O									
			GPIO0_31	7	IO	pad								
① M21		② GPMC0_DIR PADCONFIG: PADCONFIG56 0x040840E0	GPMC0_DIR	0	O		Off / Off / Off	Off / Off / Off	7	⑤ 1.8V / 3.3V	⑥ VDDSHV0 / NA	Yes	LVCMOS	PU/PD
			VOUT0_DE	1	O									
			SPI2_D0	2	IO	0								
			③ MCASP2_AXR13	④ 4	IO	0								
			EQEP1_B	4	I	0								
			TRC_DATA14	6	O									
			GPIO0_40	7	IO	pad								
			EQEP2_S	8	IO	0								
			MCAN1_TX	9	O									
N20	NA	GPMC0_OEn_REn PADCONFIG: PADCONFIG49 0x040840C4	GPMC0_OEn_REn	0	O		Off / Off / Off	Off / Off / Off	7	1.8V / 3.3V	VDDSHV0 / NA	Yes	LVCMOS	PU/PD
			VOUT0_DATA18	1	O									
			MCASP1_AXR1	2	IO	0								
			EHRPWM2_A	4	IO	0								
			SPI3_CS2	5	IO	1								
			TRC_DATA8	6	O									
			GPIO0_33	7	IO	pad								
M19	NA	GPMC0_WEn PADCONFIG: PADCONFIG50 0x040840C8	GPMC0_WEn	0	O		Off / Off / Off	Off / Off / Off	7	1.8V / 3.3V	VDDSHV0 / NA	Yes	LVCMOS	PU/PD
			VOUT0_DATA19	1	O									
			MCASP1_AXR0	2	IO	0								
			EHRPWM2_B	4	IO	0								
			SPI3_CS1	5	IO	1								
			TRC_DATA9	6	O									
			GPIO0_34	7	IO	pad								

图 2.1.1 GPIO 可复用功能

在上图中：

1. 表示 AM62L 系列的芯片引脚号（我们都是使用 ANB 只看第一行即可，第二行表示 ANQ 封装），第 M 行的 21 号引脚。
2. 在这里我们只要关注 0x040840E0 即可，这个地址就是我们在设备树需要填写的地址。
3. 表示 M21 引脚的功能复用表，这里有七种复用功能。
4. 表示 M21 引脚的功能复用的模式编号，例如需要复用成 GPIO 那么就选择 7。使用什么功能就选择那个编号即可。
5. 表示 M21 引脚支持的电压，1.8V 或者 3.3V，这个是根据电源域决定的，通常硬件固定住的，不可切换。
6. 表示 M21 引脚是在那个电源域下，这里为 VDDSHV0。

2.2 底板原理图+核心板接口数据手册查询复用功能

2.2.1 根据管脚号查找

在 ATK-DLAM62L 底板原理图中找到管脚，根据管脚号在《ATK-DLAM62L 核心板接口数据手册.xlsx》里找到对应的可复用功能。

比如在开发板原理图中找到 MMC2_DAT2 引脚，对应的核心板管脚号为：A_9，那么我们可以在《ATK-DLAM62L 核心板接口数据手册.xlsx》中找到对应的引脚。

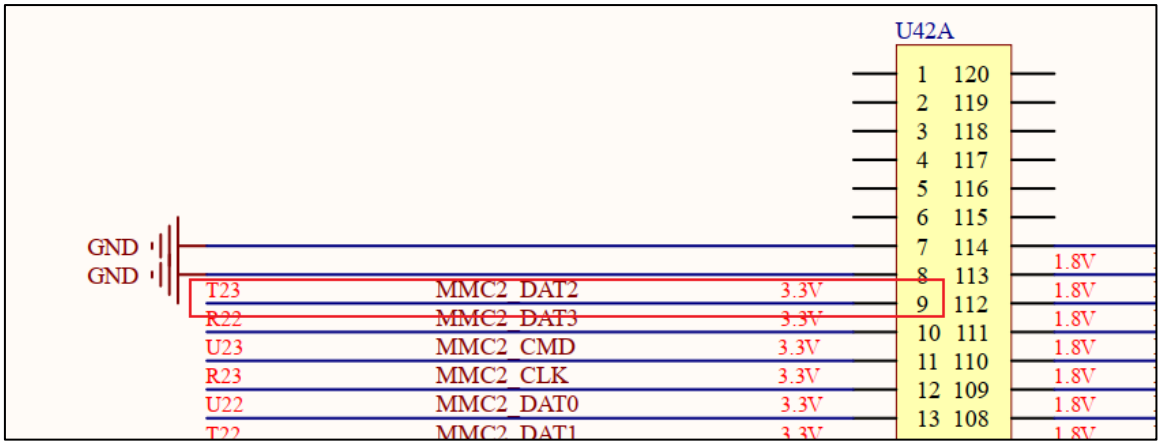


图 2.2.1 在底板原理图查找需要的管脚号

核心板管脚号	芯片引脚号	核心板默认配置	核心板信号长度	参考电平	可复用功能(需结合具体需求分析)
A_1	\				
A_2	\				
A_3	\				
A_4	\				
A_5	\				
A_6	\				
A_7		GND			
A_8		GND			
A_9	T23	MMC2_DAT2		3.3V	MMC2_DAT2\MCASP1_AXR2\UART5_TXD\GPIO0_46
A_10	R22	MMC2_DAT3		3.3V	MMC2_DAT3\MCASP1_AXR3\UART5_RXD\GPIO0_45
A_11	U23	MMC2_CMD		3.3V	MMC2_CMD\MCASP1_AFSR\MCASP1_AXR4\UART6_TXD\GPIO0_50
A_12	R23	MMC2_CLK		3.3V	MMC2_CLK\MCASP1_ACLKR\MCASP1_AXR5\UART6_RXD\GPIO0_49

图 2.2.2 在核心板接口数据手册找到对应的可复用功能

2.2.2 根据功能查找

在《ATK-DLAM62L 核心板接口数据手册.xlsx》中搜索需要的接口功能，比如 UART0_TXD，优先保留“出厂系统默认配置项”对应的管脚，因为出厂系统默认已经配置好了，可以直接使用。

1	B_40		GND			
2	B_41	B16	MCAN0_TX		3.3V	MCAN0_TX\UART5_RXD\TIMER_IO2\SYNC2_OUT\UART1_DTRn\EQEP2
3	B_42	B15	MCAN0_RX		3.3V	MCAN0_RX\UART5_TXD\TIMER_IO3\SYNC3_OUT\UART1_RIn\EQEP2_
4	B_43	B14	MCAN2_RX		3.3V	UART0_CTSn\SPI0_CS2\I2C3_SCL\UART2_RXD\TIMER_IO2\AUDIO_
5	B_44	B13	MCAN2_TX		3.3V	UART0_RTSn\SPI0_CS3\I2C3_SDA\UART2_TXD\TIMER_IO3\AUDIO_
6	B_45	C13	UART0_TXD		3.3V	UART0_TXD\ECAP2_IN_APWM_OUT\SPI2_D1\EHRPWM2_B\GPIO0_93
7	B_46	D13	UART0_RXD		3.3V	UART0_RXD\ECAP1_IN_APWM_OUT\SPI2_D0\EHRPWM2_A\GPIO0_92
8	B_47		GND			

图 2.2.3 搜索示例 1

如果出厂系统默认没有配置，再根据“可复用功能”，找到对应的管脚进行管脚复用功能修改。比如这里搜索 GPIO0_20，此管脚在“出厂系统默认配置”里没有对应的项目，意味着在出厂系统上无法直接使用此功能，需要在“可复用功能”中找到此功能，再根据此功能找到“核心板管脚名”，在设备树中进行管脚复用功能定义。

B_83		GND			
B_84	J22	VOUT0_DATA7		3.3V	GPMC0_AD7\VOUT0_DATA7\UART4_CTSn\MCASP2_AXR11\SPI1_CS1\MCASP1_AXR5\TRC_DATA5\GPIO0_22\B
B_85	H23	VOUT0_DATA6		3.3V	GPMC0_AD6\VOUT0_DATA6\UART4_RTSn\MCASP2_AXR10\SPI1_CS3\TRC_DATA4\GPIO0_21\BOOTMODE06
B_86	H22	VOUT0_DATA5		3.3V	GPMC0_AD5\VOUT0_DATA5\UART5_CTSn\MCASP2_AXR9\SPI1_CLK\TRC_DATA3\GPIO0_20\BOOTMODE05
B_87 ②	K23	VOUT0_DATA4		3.3V	GPMC0_AD4\VOUT0_DATA4\UART5_RTSn\MCASP2_AXR8\SPI1_CS0\TRC_DATA2\GPIO0_19\BOOTMODE04
B_88	J23	VOUT0_DATA3		3.3V	GPMC0_AD3\VOUT0_DATA3\UART6_CTSn\MCASP2_AXR7\SPI1_D1\TRC_DATA1\GPIO0_18\BOOTMODE03
B_89	K22	VOUT0_DATA2		3.3V	GPMC0_AD2\VOUT0_DATA2\UART6_RTSn\MCASP2_AXR6\SPI1_D0\TRC_DATA0\GPIO0_17\BOOTMODE02

图 2.2.4 搜索示例 2

① ：在“可复用功能”中找到对应的功能定义。

②：在对应的“BTB 管脚号”中知道对应的管脚名。

2.3 在出厂系统中查询管脚使用功能

开发板正常上电，打开调试串口，运行以下命令：

```
cat /sys/kernel/debug/pinctrl/4084000.pinctrl-pinctrl-single/pingroups
```

```
root@atk-dlam62l:~# cat /sys/kernel/debug/pinctrl/4084000.pinctrl-pinctrl-single/pingroups
registered pin groups:
group: usr-led-default-pins
pin 26 (PIN26)

group: main-uart0-default-pins
pin 109 (PIN109)
pin 110 (PIN110)

group: main-mcasp0-default-pins
pin 100 (PIN100)
pin 101 (PIN101)
pin 97 (PIN97)
pin 96 (PIN96)

group: wkup-i2c0-default-pins
pin 4 (PIN4)
pin 5 (PIN5)

group: main-i2c0-default-pins
pin 115 (PIN115)
pin 116 (PIN116)

group: main-user-audio-pins-default
pin 121 (PIN121)
pin 25 (PIN25)

group: main-i2c1-default-pins
pin 117 (PIN117)
pin 118 (PIN118)

group: main-i2c2-default-pins
pin 119 (PIN119)
pin 120 (PIN120)

group: main-dsi-tpcs-pins-default
pin 23 (PIN23)
pin 24 (PIN24)

group: main-epwm1-a-pins-default
pin 98 (PIN98)

group: main-epwm1-b-pins-default
pin 99 (PIN99)

group: main-uart1-default-pins
pin 102 (PIN102)
pin 103 (PIN103)
```

图 2.3.1 GPIO pingroups

可以看到每个 pin groups 中已复用的 IO。

当然也可以用如下指令看所有 GPIO 的复用功能

```
cat /sys/kernel/debug/pinctrl/4084000.pinctrl-pinctrl-single/pinmux-pins
```

但是这个指令还有 BUG，它会一直输出到 8191 的 pin 号，而芯片本身只有一百多的 GPIO，所有在输出到差不多的时候按“Ctrl+C”结束掉就行。

```
root@atk-dlam62l:~# cat /sys/kernel/debug/pinctrl/4084000.pinctrl-pinctrl-single/pinmux-pins
Pinmux settings per pin
Format: pin (name): mux_owner gpio_owner hog?
pin 0 (PIN0): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 1 (PIN1): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 2 (PIN2): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 3 (PIN3): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 4 (PIN4): 2b200000.i2c (GPIO UNCLAIMED) function wkup-i2c0-default-pins group wkup-i2c0-default-pins
pin 5 (PIN5): 2b200000.i2c (GPIO UNCLAIMED) function wkup-i2c0-default-pins group wkup-i2c0-default-pins
pin 6 (PIN6): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 7 (PIN7): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 8 (PIN8): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 9 (PIN9): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 10 (PIN10): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 11 (PIN11): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 12 (PIN12): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 13 (PIN13): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 14 (PIN14): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 15 (PIN15): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 16 (PIN16): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 17 (PIN17): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 18 (PIN18): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 19 (PIN19): gpio-keys (GPIO UNCLAIMED) function usr-button-default-pins group usr-button-default-pins
pin 20 (PIN20): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 21 (PIN21): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 22 (PIN22): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 23 (PIN23): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 24 (PIN24): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 25 (PIN25): 1-0011 (GPIO UNCLAIMED) function main-user-audio-pins-default group main-user-audio-pins-default
pin 26 (PIN26): leds (GPIO UNCLAIMED) function usr-led-default-pins group usr-led-default-pins
pin 27 (PIN27): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 28 (PIN28): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 29 (PIN29): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 30 (PIN30): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 31 (PIN31): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 32 (PIN32): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 33 (PIN33): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 34 (PIN34): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 35 (PIN35): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 36 (PIN36): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 37 (PIN37): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 38 (PIN38): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 39 (PIN39): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 40 (PIN40): (MUX UNCLAIMED) (GPIO UNCLAIMED)
```

图 2.3.2 PIN 功能复用表

这里的 pin 排序是根据芯片引脚的地址决定的。

其中可以看到 PIN19 被复用为了 gpio-keys，pinctrl 为：usr-button-default-pins。

MUX_UNCLAIMED 的 IO 则是没有被复用的 IO。

第三章 环境搭建和基础操作

3.1 开发板基础使用

开发板的基础使用请参考：开发板光盘 A 盘-基础资料→09、用户手册→ 01、测试文档→【正点原子】ATK-DLAM62L 开发板快速体验手册.pdf

3.2 出厂源码编译

出厂系统 U-Boot 和内核源码编译操作请参考：开发板光盘 A 盘-基础资料→09、用户手册→ 02、开发文档→【正点原子】ATK-DLAM62L 出厂系统 SDK 使用.pdf。

3.3 固件更新

编译完的内核、设备树等文件需要更新到开发板上进行验证，请参考：开发板光盘 A 盘-基础资料→09、用户手册→ 02、开发文档→【正点原子】ATK-DLAM62L 出厂系统 SDK 使用.pdf 的“使用技巧”章节。

第四章 出厂系统外设资源

4.1 开发板板载外设资源

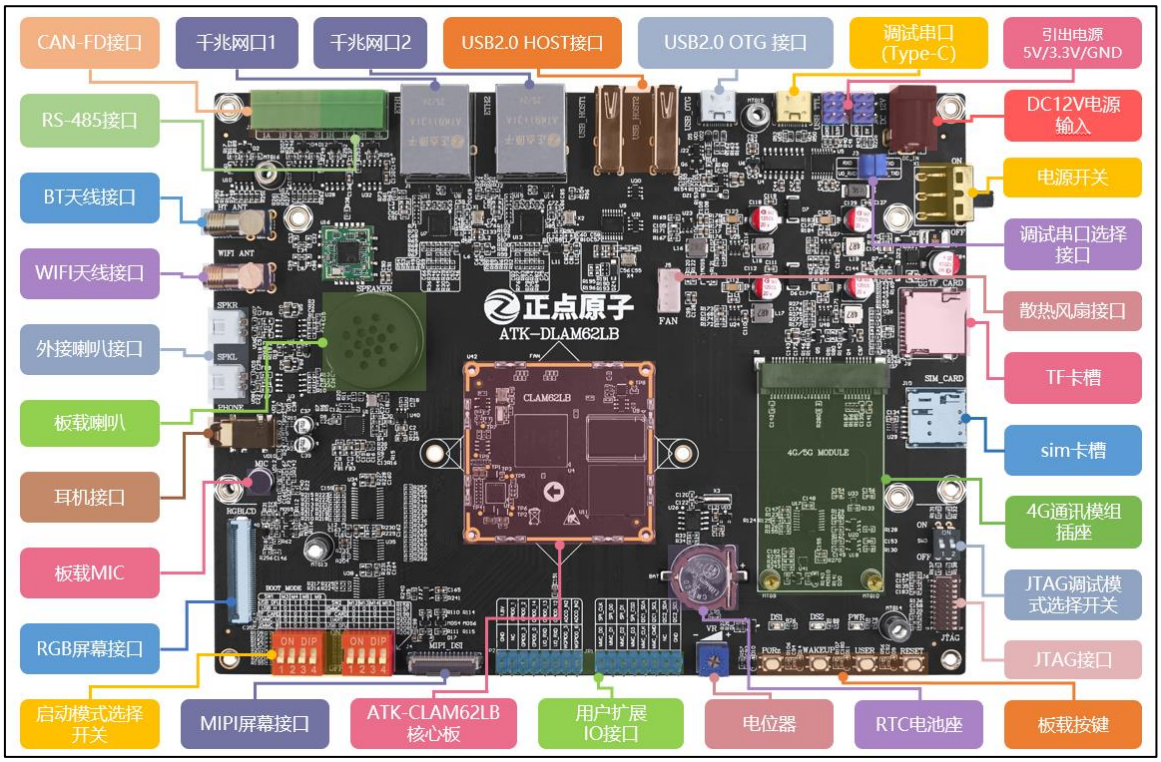


图 4.1.1 板载外设资源

更多板载外设资源说明可以查看：开发板光盘 A 盘-基础资料→09、用户手册→ 03、辅助文档→【正点原子】ATK-DLAM62L 开发板硬件参考手册.pdf。

4.2 出厂系统内核源码外设资源

内核源码版本 linux-6.12.24，在内核源码中搜索对应的 compatible 属性即可找到驱动源码。

表 4.2.1 开发板出厂 Linux 系统软件资源

类型	描述	备注
U-Boot	版本为 2025.01	提供源码
Linux Kernel	版本为 6.12.24	提供源码
Qt5	版本为 5.15.14	提供源码
交叉编译器	aarch64-buildroot-linux-gnu-gcc	用于编译 SDK 外源码如驱动、应用、QT 源码
系统烧录方法	SD 卡烧录	提供使用教程
RGB LCD	RGB 驱动	提供源码
MIPI DSI	MIPI DSI 驱动	提供源码
触摸	GT9xx 电容屏触摸屏(仅限正点原子在售)	提供源码
网络	千兆以太网 PHY 为 YT8531	提供源码

USB HOST	USB HOST 2.0	提供源码
USB OTG	USB 从机和主机	提供源码
4G 模块	高新兴 ME3630/移远 EC20	提供源码
PMIC	TPS6521402	提供源码
功能按键	两个功能按键	提供源码
RESET 按键	复位功能	提供源码
外置 RTC	PCF8563 RTC 芯片	提供源码
TF 卡/EMMC	SDMMC 驱动	提供源码
LED	GPIO	提供源码
音频	ES8390	提供源码
USB WIFI&BT	RTL8733BU	提供源码
串口	USB 调试串口、RS-485	提供源码
ePWM	LCD PWM 背光	提供源码
CAN	支持 FD CAN	提供源码
ADC	ADC 模数转换接口	提供源码

在开发板光盘 A 盘-基础资料→09、用户手册→ 02、开发文档→【正点原子】ATK-DLAM62L 嵌入式 Linux 驱动开发参考手册.pdf 和开发板光盘 A 盘-基础资料→09、用户手册→ 02、开发文档→【正点原子】ATK-DLAM62L 嵌入式 Linux C 应用编程指南.pdf 这两份教程文档中有对驱动、设备树、应用编程的相关说明。

如果项目中有以上外设需求，可以直接使用正点原子出厂系统进行开发，保留对应的外设驱动和配置。如果有其他的外设需求，需自行移植驱动和进行系统适配。

【外设复用篇说明】

本篇基于 Linux 开发板释放的管脚，进行常见的外设复用配置和实现。本文档仅供参考，具体实现效果还取决于实际硬件是否正常。

第五章 开发板 GPIO 接口复用

5.1 修改引脚为 GPIO 功能

5.1.1 添加 GPIO 功能

这里我们以 OSPI0_CSN3 为例，将该引脚配置为 GPIO0_14。

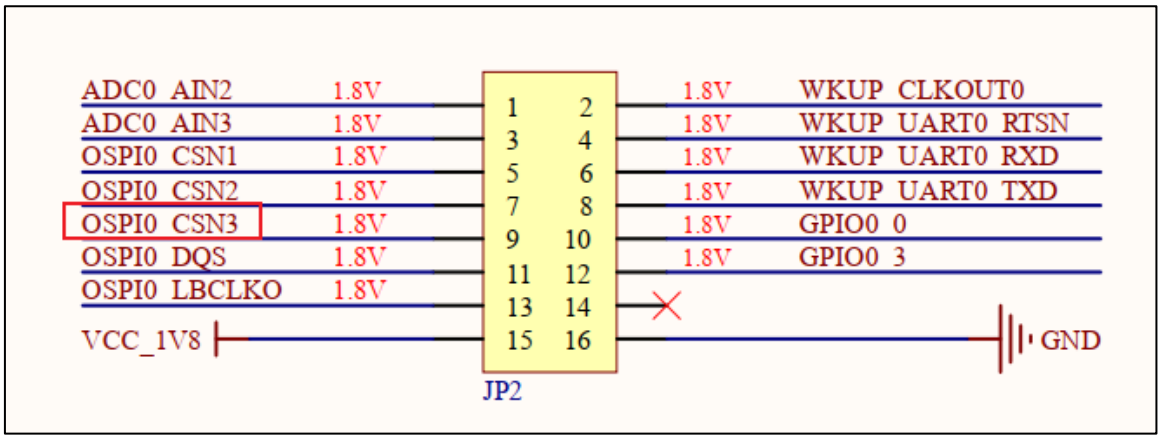


图 5.1.1 OSPI0_CSN3

5.1.2 添加 pinctrl 配置

关于 GPIO 的说明，请参考开发板光盘 A 盘-基础资料→09、用户手册→03、辅助文档→【正点原子】ATK-DLAM62L 开发板 GPIO 使用说明.pdf。

通过原理图和《ATK-DLAM62L 核心板接口数据手册.xlsx》，我们可以得知 GPIO0_14 对应 AM62L 芯片引脚号为 G23。

此时可以参考第二章的内容，从 am62l 数据手册中得到我们需要的 pinctrl 配置：基地址为 0x04084074，复用功能为 7。

ANB BALL NUMBER [1]	ANQ BALL NUMBER [1]	BALL NAME [2] PADCONFIG Register [15] PADCONFIG Address [16]	SIGNAL NAME [3]	MUX MODE [4]	TYPE [5]	DSIS [6]	B ST DU RE (RX/T
C23	NA	OSPI0_CSn3 PADCONFIG: PADCONFIG29 0x04084074	OSPI0_CSn3	0	O		Off /
			OSPI0_RESET_OUT0	1	O		
			OSPI0_ECC_FAIL	2	I	1	
			MCASP1_ACLKR	3	IO	0	
			MCASP1_AXR3	4	IO	0	
			UART5_TXD	5	O		
			ADC_EXT_TRIGGER1	6	I	0	
			GPIO0_14	7	IO	pad	
		OSPI0_D0	OSPI0_D0	0	IO	0	

图 5.1.2 C23 引脚复用功能

查询到我们需要的内容后，打开 SDK/board-support/ti-linux-kernel-6.12.24/arch/arm64/boot/dts/alientek/alientek-am62l-common.dtsi 设备树文件，在 main_sys_gpio_pins_default 节点中添加 pinctrl 配置。

AM62LX_IOPAD(0x00747, PIN_OUTPUT, 7) /* (C23) GPIO0_14 *

添加完后如下图所示：

```
&pmx0 {

    pinctrl-names = "default";
    pinctrl-0 = <&main_sys_gpio_pins_default>;

    main_sys_gpio_pins_default: main-sys-gpio-pins-default {
        pinctrl-single,pins = <
            AM62LX_IOPAD(0x003C, PIN_OUTPUT, 7) /* (D22) GPIO0_0 */
            AM62LX_IOPAD(0x0040, PIN_OUTPUT, 7) /* (E18) GPIO0_1 */
            AM62LX_IOPAD(0x0044, PIN_INPUT, 7) /* (E22) GPIO0_2 */
            AM62LX_IOPAD(0x0048, PIN_INPUT, 7) /* (C22) GPIO0_3 */
            AM62LX_IOPAD(0x0074, PIN_OUTPUT, 7) /* (C23) GPIO0_14 */
        >;
    };
```

图 5.1.3 添加 C23 引脚 pinctrl 配置

保存文件后，用 3.2 小节的配置，重新编译设备树文件。

```
make linux-dtbs
make linux_stage
```

将编译得到的 board-support/built-images 目录下的文件替换到开发板的/boot 目录后，重启开发板。

```
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$ ls
6.12.24  bl1.bin  bl31.bin  dtb  Image  tiboot3.bin  tispl.bin  u-boot.img  uEnv.txt
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$
```

图 5.1.4 board-support/built-images 目录

5.1.3 功能验证

更新完设备树并重启后，在系统中查询当前内核系统的管脚配置。

```
cat /sys/kernel/debug/pinctrl/4084000.pinctrl-pinctrl-single/pingroups
```

主要看前面配置的 GPIO 管脚功能。

```
root@atk-dlam62l:~# cat /sys/kernel/debug/pinctrl/4084000.pinctrl-pinctrl-single/pingroups
registered pin groups:
group: main-sys-gpio-pins-default
pin 15 (PIN15)
pin 16 (PIN16)
pin 17 (PIN17)
pin 18 (PIN18)
pin 29 (PIN29)
```

图 5.1.5 查看系统中管脚配置

配置成功后即可使用上述配置的 GPIO 功能。

```
gpioset 0 14=1      //设置 gpio0_14 为高电平
gpioset 0 14=0      //设置 gpio0_14 为低电平
```

第六章 UART 复用

ATK-CLAM62L 核心板常用于工业数据采集，硬件上最多支持 8 路 UART。UART 外设引脚可参考 ATK-DLAM62L 核心板接口数据手册.xlsx，也可参考 AM62L 数据手册。开发板中引出了 3 路串口，分别是调试串口（UART 0）和 RS485(UART1、UART4)下文将介绍如何将其他接口复用成串口功能，以 UART5 为例。

6.1 UART5

通过查看 ATK-DLAM62L 核心板接口数据手册.xlsx，可知，GPIO0_14 和 GPIO0_13 可以复用为 UART5 功能。

B_34	E22	GPIO0_2		1.8V	OSPI0_DQS\UART5_CTSn\GPIO0_2	
B_35	C23	GPIO0_14		1.8V	OSPI0_CSn3\OSPI0_RESET_OUT0\OSPI0_ECC_FAIL\MCASP1_ACLKR\MCASP1_AXR3\UART5_TXD\ADC_EXT_TRIGGER1\GPIO0_14	
B_36	C20	LED0		1.8V	OSPI0_CSn0\GPIO0_11	
B_37	D20	GPIO0_12		1.8V	OSPI0_CSn1\GPIO0_12	
B_38	D18	GPIO0_13		1.8V	OSPI0_CSn2\SPI1_CS1\OSPI0_RESET_OUT1\MCASP1_AFSR\MCASP1_AXR2\UART5_RXD\ADC_EXT_TRIGGER0\GPIO0_13	
B_39	E18	GPIO0_1		1.8V	OSPI0_LBCLK0\UART5_RTSn\GPIO0_1	

图 6.1.1 可复用功能

6.1.1 去掉原本复用功能

本次实验我们用的引脚是，GPIO0_13（D18）和 GPIO0_14（C23），通过查看 AM62L 数据手册，我们可以得知引脚对应的地址为 0x04084070 和 0x04084074。

接着打开 SDK/board-support/ti-linux-kernel-6.12.24/arch/arm64/boot/dts/alientek/alientek-am62l-common.dtsi 设备树文件，通过搜索引脚地址的后三位（070、074），将系统中有用到这两个引脚的地方屏蔽掉。 比如在上一章我们就用到了 GPIO0_14（C23），因此需要将其屏蔽掉。

```
alientek-am62l-common.dtsi M
arch > arm64 > boot > dts > alientek > alientek-am62l-common.dtsi
170 };
171
172 &pmx0 {
173
174     pinctrl-names = "default";
175     pinctrl-0 = <&main_sys_gpio_pins_default>;
176
177     main_sys_gpio_pins_default: main-sys-gpio-pins-default {
178         pinctrl-single,pins = <
179             AM62LX_IOPAD(0x003C, PIN_OUTPUT, 7) /* (D22) GPIO0_0 */
180             AM62LX_IOPAD(0x0040, PIN_OUTPUT, 7) /* (E18) GPIO0_1 */
181             AM62LX_IOPAD(0x0044, PIN_INPUT, 7) /* (E22) GPIO0_2 */
182             AM62LX_IOPAD(0x0048, PIN_INPUT, 7) /* (C22) GPIO0_3 */
183             /*AM62LX_IOPAD(0x0074, PIN_OUTPUT, 7)  (C23) GPIO0_14 */
184         >;
185     };
> 074   Aa ab .*   1 of 1
```

图 6.1.2 屏蔽 gpio0_14 pinctrl 配置

6.1.2 设备树中添加 UART5 配置

设备树的配置，找到“SDK/board-support/ti-linux-kernel-6.12.24/arch/arm64/boot/dts/alientek/alientek-am62l-common.dtsi”文件，添加如下代码所示：

1、 添加 uart 别名

在 aliases 节点下添加“serial7 = &main_uart5;”将 uart5 固定为 serial7，这样在系统中就可以将 uart5 固定为/dev/ttyS7。添加后如下图所示：

```
serial7 = &main_uart5;
```

```
19      compatible = "ti,am62l3-evm", "ti,am62l3";
20      model = "Texas Instruments AM62L3 Evaluation Module";
21
22      aliases {
23          mmc0 = &sdhci0;
24          mmc1 = &sdhci1;
25          serial3 = &main_uart1;
26          serial4 = &main_uart2;
27          serial5 = &main_uart3;
28          serial6 = &main_uart4;
29          serial7 = &main_uart5;
30          serial8 = &main_uart6;
31      };
```

图 6.1.3 添加 uart 别名

2、添加 uart5 的 pinctr 配置

从数据手册我可以得到引脚地址和复用功能的序号，将如下内容添加到&pmx0 节点中，添加完成后如下图所示：

```
main_uart5_pins_default: main-uart5-default-pins {
        pinctrl-single,pins = <
            AM62LX_IOPAD(0x0070, PIN_INPUT, 5) /* (D18) UART5_RXD */
            AM62LX_IOPAD(0x0074, PIN_OUTPUT, 5)    /* (C23) UART5_TXD */
        >;
        bootph-all;
    };
```

```
225     main_uart4_pins_default: main-uart4-default-pins {
226         pinctrl-single,pins = <
227             AM62LX_IOPAD(0x00ec, PIN_INPUT, 3)  /* (M23) UART4_RXD */
228             AM62LX_IOPAD(0x00f0, PIN_OUTPUT, 3) /* (M22) UART4_TXD */
229         >;
230         bootph-all;
231     };
232
233     main_uart5_pins_default: main-uart5-default-pins {
234         pinctrl-single,pins = <
235             AM62LX_IOPAD(0x0070, PIN_INPUT, 5)  /* (D18) UART5_RXD */
236             AM62LX_IOPAD(0x0074, PIN_OUTPUT, 5) /* (C23) UART5_TXD */
237         >;
238         bootph-all;
239     };
240
241     main_i2c0_pins_default: main-i2c0-default-pins {
242         pinctrl-single,pins = <
243             AM62LX_IOPAD(0x01cc, PIN_INPUT_PULLUP, 0) /* (B7) I2C0_SCL */
244             AM62LX_IOPAD(0x01d0, PIN_INPUT_PULLUP, 0) /* (A7) I2C0_SDA */
245         >;
```

图 6.1.4 添加 pinctrl 配置

3、追加 uart5 节点

添加& main_uart5，使能节点和添加默认引脚配置，添加完如下图所示：

```
&main_uart5 {
```

```
        pinctrl-names = "default";
        pinctrl-0 = <&main_uart5_pins_default>;
        status = "okay";
        bootph-all;
    };
```

```
&main_uart4 {
    pinctrl-names = "default";
    pinctrl-0 = <&main_uart4_pins_default>;
    status = "okay";
    bootph-all;
};

&main_uart5 {
    pinctrl-names = "default";
    pinctrl-0 = <&main_uart5_pins_default>;
    status = "okay";
    bootph-all;
};

&main_i2c0 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&main_i2c0_pins_default>;
    clock-frequency = <400000>;
    bootph-all;
```

图 6.1.5 添加 uart5 节点

保存文件后，用 3.2 小节的配置，重新编译设备树文件。

```
make linux-dtbs
make linux_stage
```

将编译得到的 board-support/built-images 目录下的文件替换到开发板的/boot 目录后，重启开发板。

```
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$ ls
6.12.24  bl1.bin  bl31.bin  dtb  Image  tiboot3.bin  tispl.bin  u-boot.img  uEnv.txt
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$
```

图 6.1.6 board-support/built-images 目录

6.1.3 功能测试

这里使用正点原子串口模块进行测试，硬件连接如下图所示：

开发板 gpio0_14 引脚接串口模块 RXD，开发板 gpio0_13 引脚接串口模块 TXD，串口模块 type-c 接口连接电脑。（注：这组 IO 是 1.8V 的，所有串口模块需要拨到 1.8V）

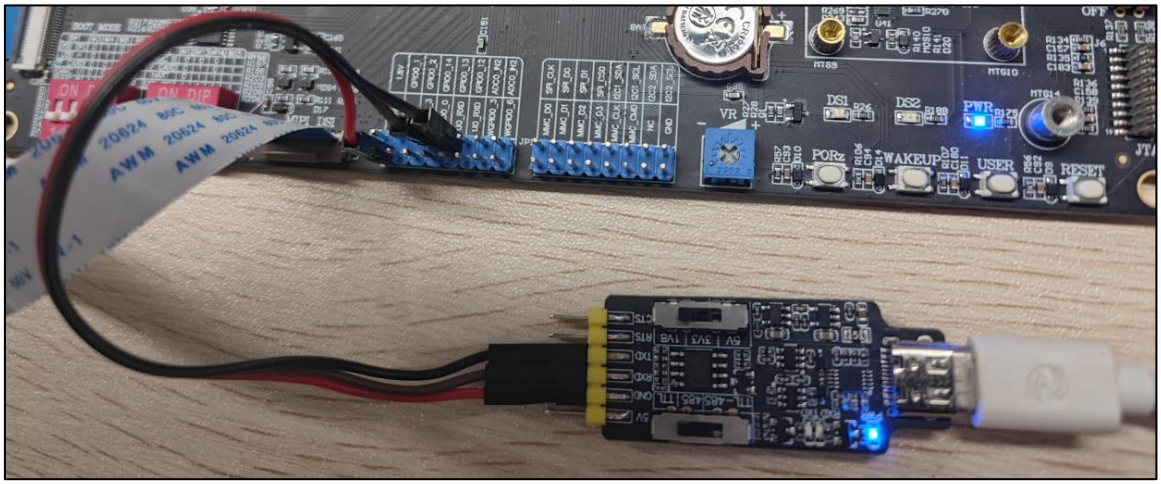


图 6.1.7 硬件连接

这里使用正点原子提供的串口上位机进行测试，位于开发板光盘 A-基础资料→03、软件→XCOM V2.0.exe。

启动开发板，通过下面的指令配置串口

```
stty -F /dev/ttyS7 ispeed 115200 ospeed 115200 cs8
```

stty 指令解释：

（1） -F(--file)：打开指定的设备,并用此设备作为输入来代替标准输入

（2） ispeed N：设置输入速率为 N

（3） ospeed N： 设置输出速率为 N

（4） csN：把字符长度设为 N

更多关于该指令用法请输入 stty -help 查看

输入如下指令等待上位机发来的数据，注意不要发中文，否则可能会乱码。按 Ctrl+c 结束接收。

```
cat /dev/ttyS7
```

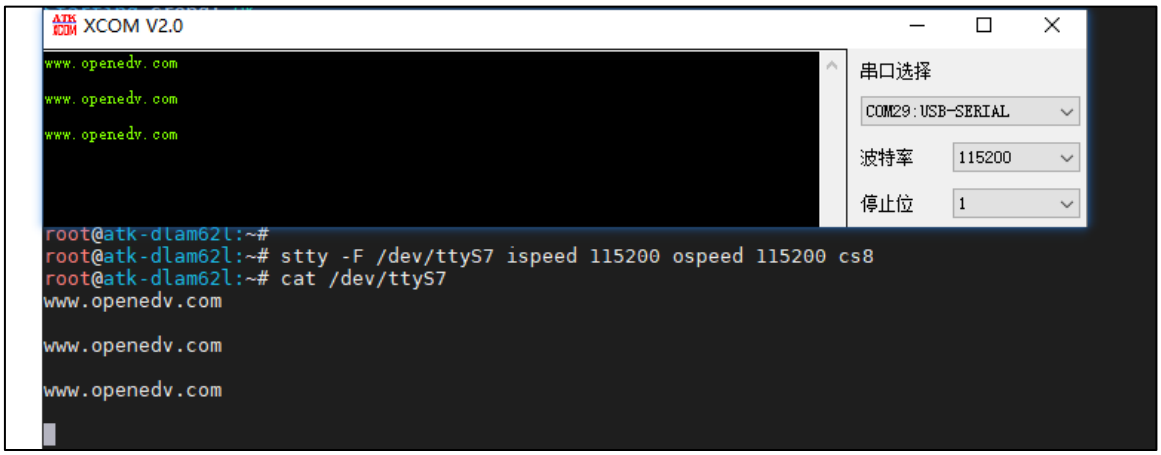


图 6.1.8 串口接收数据

使用以下命令发送数据到上位机：

```
echo www.openedv.com > /dev/ttyS7
```

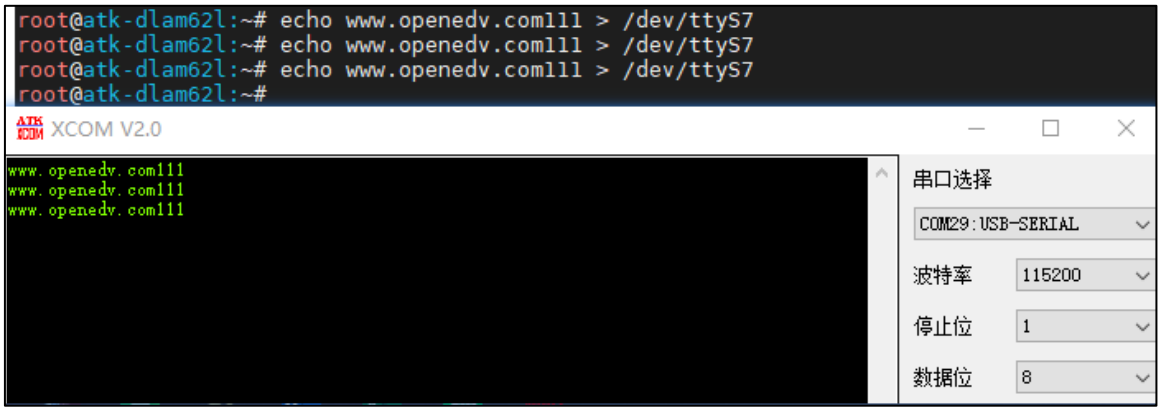


图 6.1.9 串口发送数据

测试完毕，其他串口也可以用这种方式添加复用功能。

第七章 IIC 接口复用

ATK-CLAM62L 核心板常用于工业数据采集，硬件上最多支持 5 路 I2C，其中一路为 WKUP 域 I2C，用于控制核心板上的 PMIC 因此没有将其引出到底板。I2C 外设引脚可参考 ATK-DLAM62L 核心板接口数据手册.xlsx，也可参考 AM62L 数据手册。开发板中引出了 3 路串口，分别是 I2C0、I2C1 和 I2C2。下文将介绍如何将其他接口复用成 I2C 功能，以 I2C3 为例。

7.1 I2C3

通过查看 ATK-DLAM62L 核心板接口数据手册.xlsx，可知，GPIO0_94 和 GPIO0_95 可以复用为 I2C3 功能。

163	B_42	B15	MCAN0_RX		3.3V	MCAN0_RX\UART5_TXD\TIMER_IO3\SYNC3_OUT\UART1_RIn\EQEP2_S\GPIO0_97\MCASP2_AXR1
164	B_43	B14	MCAN2_RX		3.3V	UART0_CTSn\SPI0_CS2\I2C3_SCL\UART2_RXD\TIMER_IO2\AUDIO_EXT_REFCLK0\MCAN2_RX\G
165	B_44	B13	MCAN2_TX		3.3V	UART0_RTSn\SPI0_CS3\I2C3_SDA\UART2_TXD\TIMER_IO3\AUDIO_EXT_REFCLK1\MCAN2_TX\G
166	B_45	C13	UART0_TXD		3.3V	UART0_TXD\ECAP2_IN_APWM_OUT\SPI2_D1\EHRPWM2_B\GPIO0_93
167	B_46	D13	UART0_RXD		3.3V	UART0_RXD\ECAP1_IN_APWM_OUT\SPI2_D0\EHRPWM2_A\GPIO0_92
168	B_47		GND			

图 7.1.1 可复用功能

7.1.1 去掉原本功能

本次实验我们用的引脚是，GPIO0_94(B14)和 GPIO0_95(B13)，通过查看 AM62L 数据手册，我们可以得知引脚对应的地址为 0x040841BC 和 0x040841C0。

接着打开 SDK/board-support/ti-linux-kernel-6.12.24/arch/arm64/boot/dts/alientek/alientek-am62l-common.dtsi 设备树文件，通过搜索引脚地址的后三位（1BC、1C0），将系统中有用到这两个引脚的地方屏蔽掉。比如在出厂系统中，这两个引脚默认为 CAN2 的功能，因此需要将其屏蔽掉。

```
&main_mcan2 {
    pinctrl-names = "default";
    pinctrl-0 = <&main_mcan2_pins_default>;
    status = "disable";
};
```

图 7.1.2 关闭 can2 节点

```
    main_mcan0_pins_default: main-mcan0-default-pins {
        pinctrl-single,pins = <
            AM62LX_IOPAD(0x01c8, PIN_INPUT, 0) /* (B15) MCAN0_RX */
            AM62LX_IOPAD(0x01c4, PIN_OUTPUT, 0) /* (B16) MCAN0_TX */
        >;
    };

    main_mcan2_pins_default: main-mcan2-default-pins {
        pinctrl-single,pins = <
            /*AM62LX_IOPAD(0x01bc, PIN_INPUT, 6)  (B14) UART0_CTSn.MCAN2_RX */
            /*AM62LX_IOPAD(0x01c0, PIN_OUTPUT, 6)  (B13) UART0_RTSn.MCAN2_TX */
        >;
    };

    rgmii1_pins_default: rgmii1-default-pins {
        pinctrl-single,pins = <
            AM62PX_IOPAD(0x0138, PIN_INPUT, 0) /* (Y8) RGMII1_RD0 */
```

图 7.1.3 屏蔽 can2 pinctrl 配置

7.1.2 设备树中添加 I2C3 配置

设备树的配置，找到“SDK/board-support/ti-linux-kernel-6.12.24/arch/arm64/boot/dts/alientek/alientek-am62l-common.dtsi”文件，添加如下代码所示：

1、添加 I2C3 的 pinctr 配置

从数据手册我可以得到引脚地址和复用功能的序号，将如下内容添加到&pmx0 节点中，添加完成后如下图所示：

```
main_i2c3_pins_default: main-i2c3-default-pins {
    pinctrl-single,pins = <
        AM62LX_IOPAD(0x01bc, PIN_INPUT_PULLUP, 0) /* (B14) I2C3_SCL */
        AM62LX_IOPAD(0x01c0, PIN_INPUT_PULLUP, 0) /* (B13) I2C3_SDA */
    >;
};
```

```
256
257     main_i2c2_pins_default: main-i2c2-default-pins {
258         pinctrl-single,pins = <
259             AM62LX_IOPAD(0x01dc, PIN_INPUT_PULLUP, 0) /* (B8) I2C2_SCL */
260             AM62LX_IOPAD(0x01e0, PIN_INPUT_PULLUP, 0) /* (D8) I2C2_SDA */
261         >;
262     };
263
264     main_i2c3_pins_default: main-i2c3-default-pins {
265         pinctrl-single,pins = <
266             AM62LX_IOPAD(0x01bc, PIN_INPUT_PULLUP, 0) /* (B14) I2C3_SCL */
267             AM62LX_IOPAD(0x01c0, PIN_INPUT_PULLUP, 0) /* (B13) I2C3_SDA */
268         >;
269     };
270
271     usr_led_pins_default: usr-led-default-pins {
272         pinctrl-single,pins = <
273             AM62LX_IOPAD(0x0068, PIN_OUTPUT, 7) /* (C20) GPIO0_11 */
```

图 7.1.4 添加 pinctrl 配置

2、追加 I2C3 节点

添加& main_i2c3，使能节点和添加默认引脚配置，添加完如下图所示：

```
&main_i2c3 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&main_i2c3_pins_default>;
    clock-frequency = <400000>;
};
```

```
503         pcf8563@51 {
504             status = "okay";
505             compatible = "nxp,pcf8563";
506             reg = <0x51>;
507         };
508
509     };
510
511     &main_i2c3 {
512         status = "okay";
513         pinctrl-names = "default";
514         pinctrl-0 = <&main_i2c3_pins_default>;
515         clock-frequency = <400000>;
516
517     };
518
519     &wkup_i2c0 {
520         clock-frequency = <400000>;
521         pinctrl-names = "default";
```

图 7.1.5 添加 I2C3 节点

保存文件后，用 3.2 小节的配置，重新编译设备树文件。

```
make linux-dtbs
make linux_stage
```

将编译得到的 board-support/built-images 目录下的文件替换到开发板的/boot 目录后，重启开发板。

```
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$ ls
6.12.24  bl1.bin  bl31.bin  dtb  Image  tiboot3.bin  tispl.bin  u-boot.img  uEnv.txt
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$
```

图 7.1.6 board-support/built-images 目录

7.1.3 功能测试

本章演示的引脚在开发板上以作为其它功能，在硬件上不方便演示。所有这里我们只在系统中查看是否复用成 I2C3 功能。执行如下指令可以看到验证成功：

```
cat /sys/kernel/debug/pinctrl/pinctrl-maps
```

```
group main-dsi-tpcs-pins-default
function main-dsi-tpcs-pins-default

device 20030000.i2c
state default
type MUX_GROUP (2)
controlling device 4084000.pinctrl
group main-i2c3-default-pins
function main-i2c3-default-pins

device 23010000.pwm
state default
type MUX_GROUP (2)
```

图 7.1.7 I2C3 复用功能

```
i2cdetect  -l
```

```
root@atk-dlam62l:~# i2cdetect  -l
i2c-3   i2c         OMAP I2C adapter                    I2C adapter
i2c-1   i2c         OMAP I2C adapter                    I2C adapter
i2c-4   i2c         OMAP I2C adapter                    I2C adapter
i2c-2   i2c         OMAP I2C adapter                    I2C adapter
i2c-0   i2c         OMAP I2C adapter                    I2C adapter
root@atk-dlam62l:~#
root@atk-dlam62l:~#
```

图 7.1.8 I2C3 总线

这里需要先说明一下 AM62L 的 iic 设备总线号，因为 AM62L 内部有一个 wkup 域，其中有一个 iic 总线，WKUP_IIC 用于连接到核心板上的 PMIC，在系统初始化时，它会占用一个总线，即 i2cdetect 扫出的 i2c-0。因此在 ARM 核中的 IIC 总线需要依次往后推，所以 i2c3 对应的是 i2c-4。

第八章 SPI 接口复用

ATK-CLAM62L 核心板常用于工业数据采集，硬件上最多支持 4 路 SPI。SPI 外设引脚可参考 ATK-DLAM62L 核心板接口数据手册.xlsx，也可参考 AM62L 数据手册。开发板中通过 2.54mm 排针引出了 SPI0。下文将介绍如何将其复用成 SPI 功能。

8.1 SPI0

通过查看 ATK-DLAM62L 核心板接口数据手册.xlsx，可知，E13、E12、B12 和 E11 可以复用为 SPI0 功能。

B_48	E13	SPI0_CLK		3.3V	SPI0_CLK\CP_GEMAC_CPTS0_TS_SYNC\EHRPWM1_A\GPIO0_89
B_49	E12	SPI0_D0		3.3V	SPI0_D0\CP_GEMAC_CPTS0_HW1TSPUSH\EHRPWM1_B\GPIO0_90
B_50	B12	SPI0_D1		3.3V	SPI0_D1\CP_GEMAC_CPTS0_HW2TSPUSH\EHRPWM_TZn_IN0\GPIO0_91
B_51	E11	SPI0_CS0		3.3V	SPI0_CS0\EHRPWM0_A\GPIO0_87
B_52	D11	EHRPWM0_B		3.3V	SPI0_CS1\CP_GEMAC_CPTS0_TS_COMP\EHRPWM0_B\ECAP0_IN_APWM_OUT\
B_53		GND			

图 8.1.1 可复用功能

8.1.1 去掉原本功能

本次实验我们用的引脚是，GPIO0_89(E13)、GPIO0_90(E12)、GPIO0_91(B12) 和 GPIO0_87(E11)，通过查看 AM62L 数据手册，我们可以得知引脚对应的地址为 0x040841A8、0x040841AC、0x040841B0 和 0x040841A0。

接着打开 SDK/board-support/ti-linux-kernel-6.12.24/arch/arm64/boot/dts/alientek/alientek-am62l-common.dtsi 设备树文件，通过搜索引脚地址的后三位，将系统中有用到这两个引脚的地方屏蔽掉。

8.1.2 设备树中添加 SPI0 配置

设备树的配置，找到“SDK/board-support/ti-linux-kernel-6.12.24/arch/arm64/boot/dts/alientek/alientek-am62l-common.dtsi”文件，添加如下代码所示：

1、添加 SPI0 的 pinctr 配置

从数据手册我可以得到引脚地址和复用功能的序号，将如下内容添加到&pmx0 节点中，添加完成后如下图所示：

```
spi0_pins_default: spi0-pins-default {
    bootph-all;
    pinctrl-single,pins = <
        AM62LX_IOPAD(0x01a8, PIN_OUTPUT, 0) /* (E13) SPI0_CLK */
        AM62LX_IOPAD(0x01a0, PIN_OUTPUT, 0) /* (E11) SPI0_CSn0 */
        AM62LX_IOPAD(0x01ac, PIN_INPUT, 0) /* (E12) SPI0_D0 */
        AM62LX_IOPAD(0x01b0, PIN_OUTPUT, 0) /* (B12) SPI0_D1 */
    >;
};
```

```
401     pmic_irq_pins_default: pmic-irq-default-pins {
402         pinctrl-single,pins = <
403             AM62LX_IOPAD(0x01e8, PIN_INPUT, 0) /* (C8) EXTINTn */
404         >;
405     };
406
407     spi0_pins_default: spi0-pins-default {
408         bootph-all;
409         pinctrl-single,pins = <
410             AM62LX_IOPAD(0x01a8, PIN_OUTPUT, 0) /* (E13) SPI0_CLK */
411             AM62LX_IOPAD(0x01a0, PIN_OUTPUT, 0) /* (E11) SPI0_CSn0 */
412             AM62LX_IOPAD(0x01ac, PIN_INPUT, 0) /* (E12) SPI0_D0 */
413             AM62LX_IOPAD(0x01b0, PIN_OUTPUT, 0) /* (B12) SPI0_D1 */
414         >;
415     };
416
417     wkup_i2c0_pins_default: wkup-i2c0-default-pins {
418         pinctrl-single,pins = <
419             AM62LX_IOPAD(0x0010, PIN_INPUT_PULLUP, 0) /* (AB22) WKUP_I2C0_SCL */
420             AM62LX_IOPAD(0x0014, PIN_INPUT_PULLUP, 0) /* (AA22) WKUP_I2C0_SDA */
421         >;
422     };
```

图 8.1.2 添加 pinctrl 配置

2、追加 SPI0 节点

添加& main_spi0，使能节点和添加默认引脚配置，添加完如下图所示：

```
&main_spi0 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&spi0_pins_default>;

        spidev@0 {
                spi-max-frequency = <24000000>;
                reg = <0>;
                compatible = "spidev";
                status = "okay";
        };
};
```

```
&epwm1 {
    pinctrl-names = "default";
    pinctrl-0 = <&main_epwm1_a_pins_default>;
    status = "okay";
};

&main_spi0 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&spi0_pins_default>;

        spidev@0 {
                spi-max-frequency = <24000000>;
                reg = <0>;
                compatible = "spidev";
                status = "okay";
        };
};
```

图 8.1.3 添加 SPI0 节点

在这段设备树中，在 SPI 总线下添加一个节点名为 "spidev@0"的子节点。该设备使用通用 SPI 设备驱动 spidev。spidev 是 Linux 提供的通用 SPI 用户空间接口。驱动文件为：drivers/spi/spidev.c。用于测试 spi 通信。

保存文件后，用 3.2 小节的配置，重新编译设备树文件。

```
make linux-dtbs
make linux_stage
```

将编译得到的 board-support/built-images 目录下的文件替换到开发板的/boot 目录后，重启开发板。

```
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$ ls
6.12.24  bl1.bin  bl31.bin  dtb  Image  tiboot3.bin  tispl.bin  u-boot.img  uEnv.txt
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$
```

图 8.1.4 board-support/built-images 目录

将编译出来的镜像更新到开发板上，重启开发板。启动之后执行 lsmod 可以看到 spi 控制器驱动和 spidev 驱动都已被加载上。

```
root@atk-dlam62l:~# lsmod
Module                  Size  Used by    Not tainted
8021q                  28672  0
garp                   12288  1 8021q
mrp                    20480  1 8021q
stp                    12288  1 garp
llc                    16384  2 garp,stp
af_alg                 24576  0
ipv6                  475136 30 [permanent]
ti_am335x_adc          16384  0
spidev                 20480  0
kfifo_buf              12288  1 ti_am335x_adc
8733bu                1859584  0
rtk_btusb              69632  0
bluetooth             512000 16 rtk_btusb
cfg80211              417792  1 8733bu
ecdh_generic           16384  2 bluetooth
ecc                    36864  1 ecdh_generic
crct10dif_ce           12288  1
rfkill                 28672  4 bluetooth,cfg80211
pwm_fan                16384  0
k3_j72xx_bandgap       12288  0
rti_wdt                12288  0
dthev2                 24576  0
md5                    12288  1 dthev2
m_can_platform         12288  0
m_can                  32768  1 m_can_platform
can_dev                40960  1 m_can
ti_am335x_tscadc       12288  1 ti_am335x_adc
spi_omap2_mcspi        20480  0
root@atk-dlam62l:~#
```

图 8.1.5 lsmod

8.1.3 功能测试

本小节的测试我们会用到 spidev_test 工具。

spidev_test 是一个常用的 SPI 测试工具，允许用户测试 SPI 通信。用户可以使用它来发送和接收数据，以确保 SPI 接口和设备工作正常。

出厂系统中，默认没有这个工具，需要我们从 buildroot 配置文件中添加改工具。

在 SDK 下执行如下指令，添加 buildroot 配置。

```
make buildroot_menuconfig
```

选中 spidev，如下图所示：

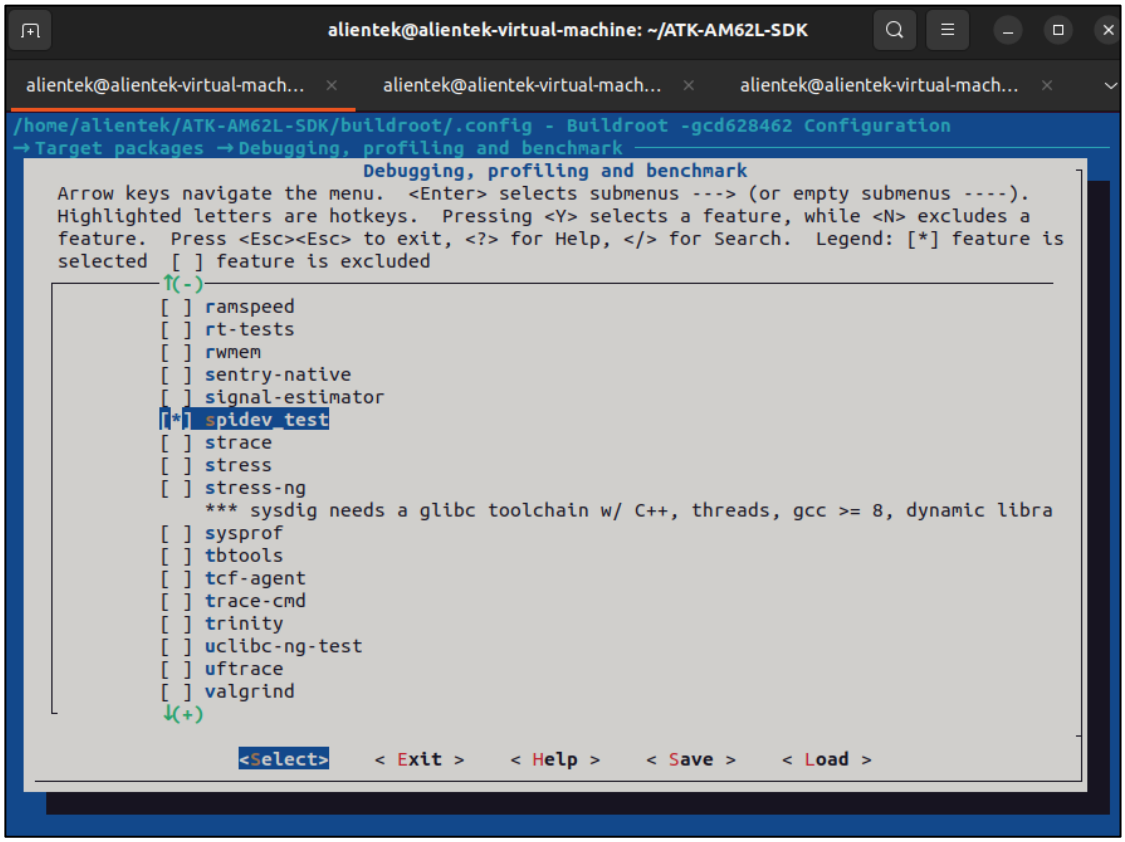


图 8.1.6 勾选 spidev_test

当我们配置完成保存，会默认修改 buildroot-external-TI/configs/ti_release_am62lx_evm_defconfig 文件。

保存后，参考第三章的内容，重新编译 SDK，打包固件，最后重新烧录系统到开发板。开发板重启后就可以使用 spidev_test。

```
root@atk-dlam62l:~# spidev_test --help
spidev_test: unrecognized option '--help'
Usage: spidev_test [-DsbdlHOLC3vpNR24SI]
  -D --device   device to use (default /dev/spidev1.1)
  -s --speed    max speed (Hz)
  -d --delay    delay (usec)
  -b --bpw      bits per word
  -i --input    input data from a file (e.g. "test.bin")
  -o --output   output data to a file (e.g. "results.bin")
  -l --loop     loopback
  -H --cpha     clock phase
  -O --cpol     clock polarity
  -L --lsb      least significant bit first
  -C --cs-high  chip select active high
  -3 --3wire    SI/SO signals shared
  -v --verbose  Verbose (show tx buffer)
  -p            Send data (e.g. "1234\xde\xad")
  -N --no-cs    no chip select
  -R --ready    slave pulls low to pause
  -2 --dual     dual transfer
  -4 --quad     quad transfer
  -8 --octal    octal transfer
  -S --size     transfer size
  -I --iter     iterations

root@atk-dlam62l:~#
```

图 8.1.7 spidev_test 说明

上一小节的配置中，我们新增了一个 spidev 设备，内核默认已经将 SPI 驱动编译到内核中，我们现在需要启动开发板，将我们创建的 SPI 设备绑定 SPI 驱动来使用。

```
ls /sys/bus/spi/devices
echo spidev > /sys/bus/spi/devices/spi0.0/driver_override
echo spi0.0 > /sys/bus/spi/drivers/spidev/bind
```

硬件上，将开发板的 SPI_D0 引脚和 SPI_D1 引脚短接，作用是进行回环测试，这种测试方法可以用来验证 SPI 接口的发送和接收功能是否正常工作，而不需要连接外部设备。

硬件接线如下如图所示：

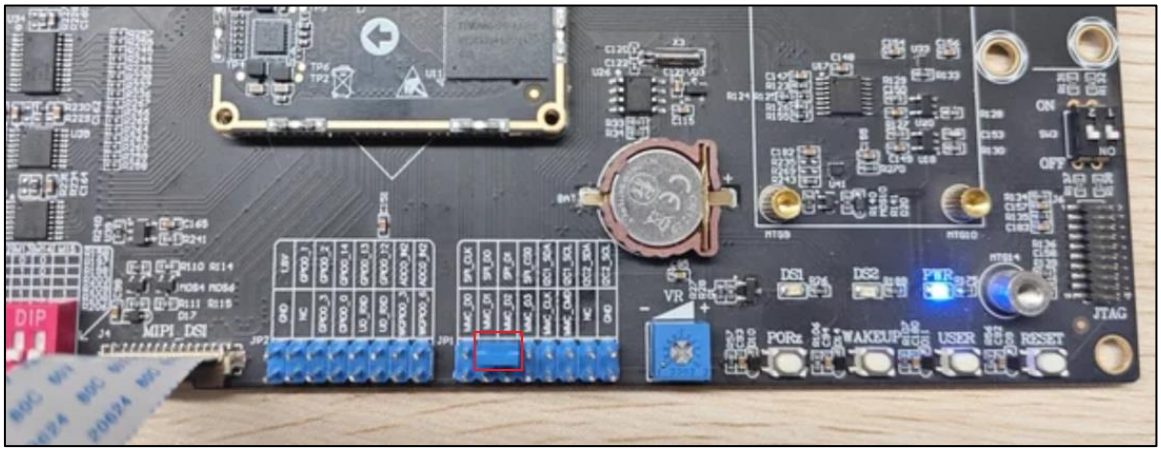


图 8.1.8 短接 SPI_D0 引脚和 SPI_D1 引脚

接好硬件后，将通过 /dev/spidev0.0 设备以 500,000 Hz 的速度发送字符串 "Hello, SPI!"，每个字节为 8 位。

```
spidev_test -D /dev/spidev0.0 -s 500000 -b 8 -p "Hello,SPI!" -v
```

```
root@atk-dlam62l:~# spidev_test -D /dev/spidev0.0 -s 500000 -b 8 -p "Hello,SPI!" -v
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 kHz)
TX | 48 65 6C 6C 6F 2C 53 50 49 21 __ __ __
```

第九章 PWM 接口复用

ATK-CLAM62L 核心板常用于工业数据采集，硬件上有 3 个 PWM 控制器，每个控制器有 2 个通道。PWM 外设引脚可参考 ATK-DLAM62L 核心板接口数据手册.xlsx，也可参考 AM62L 数据手册。开发板中引出了 3 路 PWM，分别是 RGB 屏幕背光（EHRPWM0_B）、MIPI 屏幕背光(EHRPWM1_B) 和散热风扇 PWM 控制引脚（EHRPWM1_A）。下文将介绍如何将其他接口复用成 PWM 功能，以 EHRPWM0_A 为例。

9.1 EHRPWM0_A

通过查看 ATK-DLAM62L 核心板接口数据手册.xlsx，可知，GPIO0_87（E11）可以复用为 EHRPWM0_A 功能。

B_50	B12	SPI0_D1		3.3V	SPI0_D1\CP_GEMAC_CPTS0_HW2TSPUSH\EHRPWM_TZn_IN0\GPIO0_91
B_51	E11	SPI0_CS0		3.3V	SPI0_CS0\EHRPWM0_A\GPIO0_87
B_52	D11	EHRPWM0_B		3.3V	SPI0_CS1\CP_GEMAC_CPTS0_TS_COMP\EHRPWM0_B\ECAP0_IN_APWM_OUT\

图 9.1.1 可复用功能

9.1.1 去掉原本功能

本次实验我们用的引脚是， GPIO0_87(E11)，通过查看 AM62L 数据手册，我们可以得知引脚对应的地址为 0x040841A0。

接着打开 SDK/board-support/ti-linux-kernel-6.12.24/arch/arm64/boot/dts/alientek/alientek-am62l-common.dtsi 设备树文件，通过搜索引脚地址的后三位，将系统中有用到这两个引脚的地方屏蔽掉。在上一章中，我们复用 SPI 功能时用到了这个引脚，因此要将其屏蔽掉。

```
spi0_pins_default: spi0-pins-default {
    bootph-all;
    pinctrl-single,pins = <
        AM62LX_IOPAD(0x01a8, PIN_OUTPUT, 0) /* (E13) SPI0_CLK */
        /*AM62LX_IOPAD(0x01a0, PIN_OUTPUT, 0)  (E11) SPI0_CSn0 */
        AM62LX_IOPAD(0x01ac, PIN_INPUT, 0) /* (E12) SPI0_D0 */
        AM62LX_IOPAD(0x01b0, PIN_OUTPUT, 0) /* (B12) SPI0_D1 */
    >;
};
```

图 9.1.2 屏蔽 E11 的配置

```
&main_spi0 {
        status = "disable";
        pinctrl-names = "default";
        pinctrl-0 = <&spi0_pins_default>;

        spidev@0 {
                spi-max-frequency = <24000000>;
                reg = <0>;
                compatible = "spidev";
                status = "okay";
        };
};
```

图 9.1.3 关掉 spi0 节点

9.1.2 设备树中添加 EHRPWM0_A 配置

设备树的配置，找到“SDK/board-support/ti-linux-kernel-6.12.24/arch/arm64/boot/dts/alientek/alientek-am62l-common.dtsi”文件，添加如下代码所示：

1、添加 EHRPWM0_A 的 pinctr 配置

从数据手册我可以得到引脚地址和复用功能的序号，将如下内容添加到&pmx0 节点中，添加完成后如下图所示：

```
main_epwm0_pins_default: main-epwm0-pins-default {
    pinctrl-single,pins = <
        AM62LX_IOPAD(0x01a0, PIN_OUTPUT, 2) // (E11) SPI0_CS0.EHRPWM0_A
    >;
};
```

```
172  &pmx0 {
173
174      pinctrl-names = "default";
175      pinctrl-0 = <&main_sys_gpio_pins_default>;
176
177      main_sys_gpio_pins_default: main-sys-gpio-pins-default {
178          pinctrl-single,pins = <
179              AM62LX_IOPAD(0x003C, PIN_OUTPUT, 7) /* (D22) GPIO0_0 */
180              AM62LX_IOPAD(0x0040, PIN_OUTPUT, 7) /* (E18) GPIO0_1 */
181              AM62LX_IOPAD(0x0044, PIN_INPUT, 7) /* (E22) GPIO0_2 */
182              AM62LX_IOPAD(0x0048, PIN_INPUT, 7) /* (C22) GPIO0_3 */
183              /*AM62LX_IOPAD(0x0074, PIN_OUTPUT, 7)  (C23) GPIO0_14 */
184          >;
185      };
186
187      main_epwm0_pins_default: main-epwm0-pins-default {
188          pinctrl-single,pins = <
189              AM62LX_IOPAD(0x01a0, PIN_OUTPUT, 2) // (E11) SPI0_CS0.EHRPWM0_A
190          >;
191      };
192
193
194      extcon_pins_default: extcon-pins-default {
195          pinctrl-single,pins = <
```

图 9.1.4 添加 pinctrl 配置

2、追加 epwm0 节点

添加& epwm0，使能节点和添加默认引脚配置，添加完如下图所示：

```
&epwm0 {
    pinctrl-names = "default";
    pinctrl-0 = <&main_epwm0_pins_default>;
    status = "okay";
};
```

```
742  &epwm1 {
743      pinctrl-names = "default";
744      pinctrl-0 = <&main_epwm1_a_pins_default>;
745      status = "okay";
746  };
747
748  &epwm0 {
749      pinctrl-names = "default";
750      pinctrl-0 = <&main_epwm0_pins_default>;
751      status = "okay";
752  };
753
754  &main_spi0 {
755          status = "disable";
756          pinctrl-names = "default";
757          pinctrl-0 = <&spi0_pins_default>;
```

图 9.1.5 添加 epwm0 节点

保存文件后，用 3.2 小节的配置，重新编译设备树文件。

```
make linux-dtbs
make linux_stage
```

将编译得到的 board-support/built-images 目录下的文件替换到开发板的/boot 目录后，重启开发板。

```
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$ ls
6.12.24  bl1.bin  bl31.bin  dtb  Image  tiboot3.bin  tispl.bin  u-boot.img  uEnv.txt
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$
```

图 9.1.6 board-support/built-images 目录

9.1.3 功能测试

使用 sysfs 方式操作 PWM

PWM 通过 sysfs 方式进行操控，进入到/sys/class/pwm 目录下，如下所示：

```
cd /sys/class/pwm
```

```
root@atk-dlam62l:~# cd /sys/class/pwm
root@atk-dlam62l:/sys/class/pwm# ls
pwmchip0  pwmchip1
root@atk-dlam62l:/sys/class/pwm#
```

图 9.1.7 /sys/class/pwm 目录下的内容

这里列举出了文件夹 pwmchip0，这其实就对应了 ATK-DLAM62L 的 1 个 PWM 控制器。

我们进入到 pwmchip0 目录下：

```
cd pwmchip0
```

```
root@atk-dlam62l:/sys/class/pwm# cd pwmchip0
root@atk-dlam62l:/sys/class/pwm/pwmchip0# ls
device     export     npwm     power     subsystem  uevent     unexport
root@atk-dlam62l:/sys/class/pwm/pwmchip0#
```

图 9.1.8 pwmchip0 目录下的内容

在这个目录下我们重点关注的是 export、npwm 以及 unexport 这三个属性文件，下面一一进行介绍：

- **npwm**：这是一个只读属性，读取该文件可以得知该 PWM 控制器下共有几路 PWM 输出，如下所示：

```
root@atk-dlam62l:/sys/class/pwm/pwmchip0# cat npwm
2
root@atk-dlam62l:/sys/class/pwm/pwmchip0#
```

图 9.1.9 读取 npwm 属性文件

所以由此可知，该 PWM 控制器一共有 2 路 PWM 输出，通过查阅 AM62L 参考手册也可知，PWM 确实支持 2 通道 PWM 输出。

- **export**：与 GPIO 控制一样，在使用 PWM 之前，也需要将其导出，通过 export 属性进行导出，以下所示：(0 对应的 A 通道，1 对应的 B 通道)

```
echo 0 > export
```

```
root@atk-dlam62l:/sys/class/pwm/pwmchip0# echo 0 > export
root@atk-dlam62l:/sys/class/pwm/pwmchip0# ls
device      export      npwm        power       pwm0        subsystem   uevent      unexport
root@atk-dlam62l:/sys/class/pwm/pwmchip0#
```

图 9.1.10 导出 EHRPWM0_A

数字 0 表示一个编号，注意，每个 PWM 控制器（pwmchipX）下，使用 export 属性文件导出 PWM 通道时，编号都是从 0 开始，导出成功后会在 pwmchipX（X 表示数字 0~1）目录下生成一个名为 pwm0 的目录，如图所示。

- **unexport**：将导出的 PWM 通道删除。当使用完 PWM 之后，我们需要将导出的 PWM 通道删除，譬如：

```
echo 0 > unexport
```

写入到 unexport 文件中的编号与写入到 export 文件中的编号是相对应的；需要注意的是，export 文件和 unexport 文件都是只写的、没有读权限。

如何控制 PWM

上例中通过 export 导出之后，便会生成 pwm0 这个目录，我们进入到该目录下看看：

```
root@atk-dlam62l:/sys/class/pwm/pwmchip0# cd pwm0/
root@atk-dlam62l:/sys/class/pwm/pwmchip0/pwm0# ls
capture     duty_cycle  enable      period      polarity    power       uevent
root@atk-dlam62l:/sys/class/pwm/pwmchip0/pwm0#
```

图 9.1.11 pwm0 目录下的内容

该目录下也有一些属性文件，我们重点关注 duty_cycle、enable、period 以及 polarity 这四个属性文件，接下来一一进行介绍。

很多 SoC 的 PWM 外设其硬件上并不支持极性配置，所以对应的驱动程序中并未实现这个接口，应用层自然也就无法通过 polarity 属性文件对 PWM 极性进行配置。

- **period**：用于配置 PWM 周期，可读可写；写入一个字符串数字值，以 ns（纳秒）为单位，譬如配置 PWM 周期为 10us（微秒）：

```
echo 10000 > period          #PWM 周期设置为 10us（10 * 1000ns）
```

- **duty_cycle**：用于配置 PWM 的占空比，可读可写；写入一个字符串数字值，同样也是以 ns 为单位，譬如：

```
echo 5000 > duty_cycle       #PWM 占空比设置为 5us
```

- **polarity**：用于设置极性，可读可写，可写入的值如下：

"normal"：普通；

"inversed"：反转；

```
echo normal > polarity          #默认极性
echo inversed > polarity        #极性反转
```

- **enable：**可读可写，写入"0"表示禁止 PWM；写入"1"表示使能 PWM。读取该文件获取 PWM 当前是禁止还是使能状态。

```
echo 0 > enable                 #禁止 PWM 输出
echo 1 > enable                 #使能 PWM 输出
```

通常配置好 PWM 之后，再使能 PWM。

讲解完后我们看看实验现象，将示波器接到开发板的 JP1_8 引脚和 GND，使用示波器查看 PWM 波形。

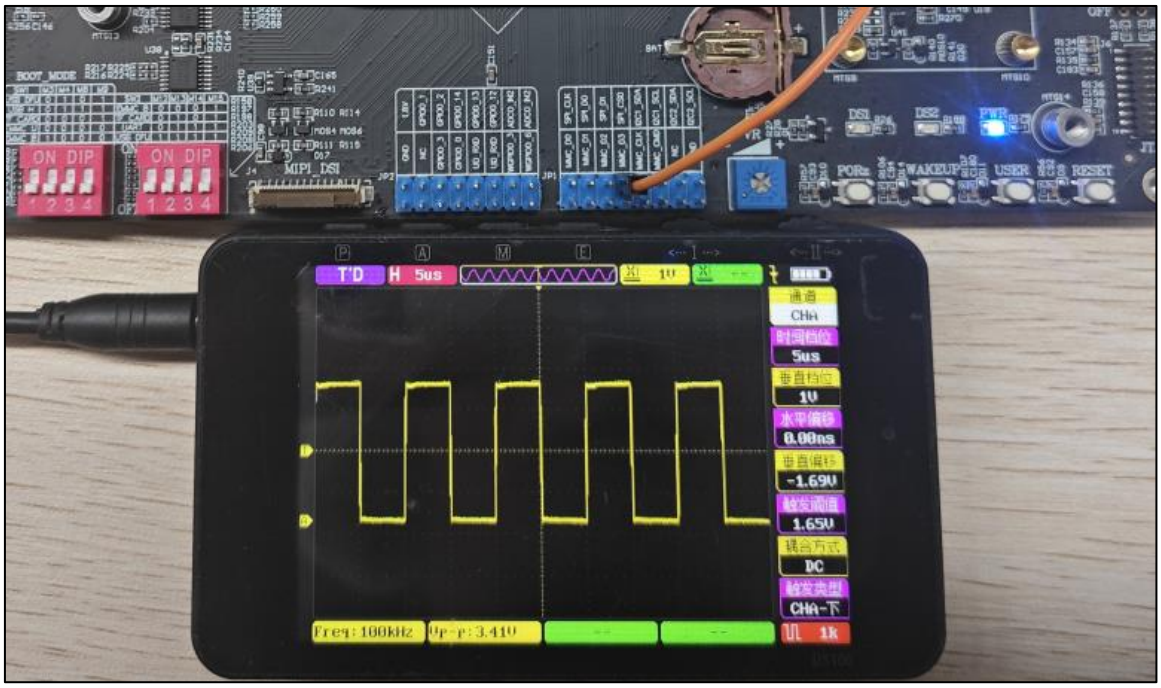


图 9.1.12 实验现象

第十章 ADC 接口复用

ATK-CLAM62L 核心板常用于工业数据采集，硬件上有 1 个 ADC 控制器，控制器有 4 个通道。该 ADC 可以作为一个 4 线电阻屏触摸采集接口，也可以分别做为 4 路 ADC 采集引脚。ADC 外设引脚可参考 ATK-DLAM62L 核心板接口数据手册.xlsx，也可参考 AM62L 数据手册。开发板中引出了 2 路 ADC，分别是 MIPI 的 ADC 检测引脚(ADC0_AIN0) 和连接到电位器检测模拟电压引脚（ADC0_AIN1）。另外 2 路 ADC 通过 2.54mm 排针引出。

10.1 ADC

出厂系统中，已经将 4 个通道的 ADC 都使能了，可执行下列命令查看 ADC 的原始值：

```
cat /sys/bus/iio/devices/iio\:device0/in_voltage1_raw
```

in_voltage1_raw 对应 ADC 通道 1 的原始数值，如果需要测 2 通道则是 in_voltage2_raw。

```
root@atk-dlam62l:~# cat /sys/bus/iio/devices/iio\:device0/in_voltage1_raw
2536
root@atk-dlam62l:~# cat /sys/bus/iio/devices/iio\:device0/in_voltage1_raw
2535
root@atk-dlam62l:~# cat /sys/bus/iio/devices/iio\:device0/in_voltage1_raw
2533
root@atk-dlam62l:~# cat /sys/bus/iio/devices/iio\:device0/in_voltage1_raw
2535
```

图 10.1.1 ADC 原始数据

ADC 电压的计算公式为：

```
D = Vin * (2^n - 1) / Vref
```

解释：

D = ADC 原始数值

Vin = 输入电压值

N = 分辨率，AM62L 的 ADC 分辨率为 12

Vref = 参考电压值，即 1.8V

第十一章 CAN 接口复用

ATK-CLAM62L 核心板常用于工业数据采集，硬件上有 3 路 CAN-FD 接口。CAN 外设引脚可参考 ATK-DLAM62L 核心板接口数据手册.xlsx，也可参考 AM62L 数据手册。开发板中引出了 2 路 CAN，分别是 CAN0 和 CAN2。下文将介绍如何将其他接口复用成 CAN 功能，以 CAN1 为例。

11.1 CAN1

通过查看 ATK-DLAM62L 核心板接口数据手册.xlsx，可知，GPIO0_38(N22)和 GPIO0_40(M21)可以复用为 CAN1 功能。

M21	VOUT0_DE		3.3V	GPMC0_DIR\VOUT0_DE\SPI2_D0\MCASP2_AXR13\EQEP1_B\TRC_DATA14\GPIO0_40\EQEP2_S\MCAN1_TX
L20	VOUT0_VSYNC		3.3V	GPMC0_CSn0\VOUT0_VSYNC\SPI2_D1\MCASP2_AXR14\EQEP1_S\TRC_DATA15\GPIO0_41
	GND			
N22	VOUT0_DATA23		3.3V	GPMC0_WAIT1\VOUT0_DATA23\SPI2_CS1\UART6_RXD\EQEP0_I\SPI3_D1\GPIO0_38\EQEP2_I\MCAN1_RX
N23	VOUT0_DATA22		3.3V	GPMC0_WAIT0\VOUT0_DATA22\MCASP1_AFSX\EQEP0_S\SPI3_D0\TRC_DATA12\GPIO0_37
P22	VOUT0_DATA21		3.3V	GPMC0_BE1n\VOUT0_DATA21\MCASP2_AXR12\EQEP0_B\SPI3_CLK\TRC_DATA11\GPIO0_36
P23	VOUT0_DATA20		3.3V	GPMC0_BE0n_CLE\VOUT0_DATA20\MCASP1_ACLKX\EQEP0_A\SPI3_CS0\TRC_DATA10\GPIO0_35
M19	VOUT0_DATA19		3.3V	GPMC0_Wen\VOUT0_DATA19\MCASP1_AXR0\EHRPWM2_B\SPI3_CS1\TRC_DATA9\GPIO0_34

图 11.1.1 可复用功能

11.1.1 去掉原本功能

本次实验我们用的引脚是，GPIO0_38(N22)和 GPIO0_40(M21)，通过查看 AM62L 数据手册，我们可以得知引脚对应的地址为 0x040840D8 和 0x040840E0。

接着打开 SDK/board-support/ti-linux-kernel-6.12.24/arch/arm64/boot/dts/alientek/alientek-am62l-common.dtsi 设备树文件，通过搜索引脚地址的后三位，将系统中有用到这两个引脚的地方屏蔽掉。

在出厂系统中，这两个引脚以被复用为 RGB 屏幕的功能，但是默认并没有开启 RGB 屏幕功能。这里我们只需要确保出厂系统中/boot/udev.txt 文件中的 RGB 屏幕扩展设备树被屏蔽掉了即可。

```
# This uEnv.txt file can contain additional environment settings that you
# want to set in U-Boot at boot time.  This can be simple variables such
# as the serverip or custom variables.  The format of this file is:
#    variable=value
# NOTE: This file will be evaluated after the bootcmd is run and the
#       bootcmd must be set to load this file if it exists (this is the
#       default on all newer U-Boot images.  This also means that some
#       variables such as bootdelay cannot be changed by this file since
#       it is not evaluated until the bootcmd is run.

#name_overlays=ti/alientek-am62l-mipi-10p1-800p.dtbo
#name_overlays=ti/alientek-am62l-mipi-5p5-1080p.dtbo
name_overlays=ti/alientek-am62l-mipi-5p5-720p.dtbo

#name_overlays=ti/alientek-am62l-rgb-10p1-800p.dtbo
#name_overlays=ti/alientek-am62l-rgb-4p3-272p.dtbo
#name_overlays=ti/alientek-am62l-rgb-4p3-480p.dtbo
#name_overlays=ti/alientek-am62l-rgb-7-480p.dtbo
#name_overlays=ti/alientek-am62l-rgb-7-600p.dtbo
~
```

图 11.1.2 屏蔽 RGB 屏扩展设备树

11.1.2 设备树中添加 CAN1 配置

设备树的配置，找到“SDK/board-support/ti-linux-kernel-6.12.24/arch/arm64/boot/dts/alientek/alientek-am62l-common.dtsi”文件，添加如下代码所示：

1、添加 CAN1 的 pinctr 配置

从数据手册我可以得到引脚地址和复用功能的序号，将如下内容添加到&pmx0 节点中，添加完成后如下图所示：

```
main_mcan1_pins_default: main-mcan1-default-pins {
        pinctrl-single,pins = <
            AM62LX_IOPAD(0x00d8, PIN_INPUT, 9) /* (N22) MCAN1_RX */
            AM62LX_IOPAD(0x00e0, PIN_OUTPUT, 9) /* (M21) MCAN1_TX */
        >;
    };
```

```
337     main_mcan0_pins_default: main-mcan0-default-pins {
338         pinctrl-single,pins = <
339             AM62LX_IOPAD(0x01c8, PIN_INPUT, 0) /* (B15) MCAN0_RX */
340             AM62LX_IOPAD(0x01c4, PIN_OUTPUT, 0) /* (B16) MCAN0_TX */
341         >;
342     };
343
344     main_mcan1_pins_default: main-mcan1-default-pins {
345         pinctrl-single,pins = <
346             AM62LX_IOPAD(0x00d8, PIN_INPUT, 9) /* (N22) MCAN1_RX */
347             AM62LX_IOPAD(0x00e0, PIN_OUTPUT, 9) /* (M21) MCAN1_TX */
348         >;
349     };
350
351     main_mcan2_pins_default: main-mcan2-default-pins {
352         pinctrl-single,pins = <
353             AM62LX_IOPAD(0x01bc, PIN_INPUT, 6)  /*(B14) UART0_CTSn.MCAN2_RX */
354             AM62LX_IOPAD(0x01c0, PIN_OUTPUT, 6)  /*(B13) UART0_RTSn.MCAN2_TX */
355         >;
356     };
```

图 11.1.3 添加 pinctrl 配置

2、追加 CAN1 节点

添加& main_mcan1，使能节点和添加默认引脚配置，添加完如下图所示：

```
&main_mcan1 {
    pinctrl-names = "default";
    pinctrl-0 = <&main_mcan1_pins_default>;
    status = "okay";
};
```

```
&main_mcan0 {
    pinctrl-names = "default";
    pinctrl-0 = <&main_mcan0_pins_default>;
    status = "okay";
};

&main_mcan1 {
    pinctrl-names = "default";
    pinctrl-0 = <&main_mcan1_pins_default>;
    status = "okay";
};

&main_mcan2 {
    pinctrl-names = "default";
    pinctrl-0 = <&main_mcan2_pins_default>;
    status = "okay";
};
```

图 11.1.4 添加 can1 节点

保存文件后，用 3.2 小节的配置，重新编译设备树文件。

```
make linux-dtbs
make linux_stage
```

将编译得到的 board-support/built-images 目录下的文件替换到开发板的/boot 目录后，重启开发板。

```
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$ ls
6.12.24  bl1.bin  bl31.bin  dtb  Image  tiboot3.bin  tispl.bin  u-boot.img  uEnv.txt
alientek@alientek-virtual-machine:~/ATK-AM62L-SDK/board-support/built-images$
```

图 11.1.5 board-support/built-images 目录

11.1.3 功能测试

本章演示的引脚在开发板上以作为其它功能，在硬件上不方便演示。所有这里我们只在系统中查看是否复用成 CAN1 功能。执行如下指令可以看到验证成功：

```
ifconfig -a
```

```
root@atk-dlam62l:~# ifconfig -a
can0      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:11

can1      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:12

can2      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:13
```

图 11.1.6 can 节点

可以看到三个 can 都已经被使能，其中 can1 就是我们复用成功的 can1 节点。